

Generator-based for loop

Generator ranges without coroutine overhead

Document #: P2881R0
Date: 2023-05-18
Project: Programming Language C++
Audience: EWG
EWGI
Reply-to: Jonathan Müller
<jmueller@think-cell.com>
Barry Revzin
<barry.revzin@gmail.com>

Contents

- 1 Abstract
- 2 Motivation
 - 2.1 Performance of `std::generator`, coroutines, and iterators
 - 2.2 Inability to `co_yield` from nested scope
 - 2.3 Limitation to a single type
- 3 Generator Range Idiom
- 4 Proposed Design
 - 4.1 `std::control_flow`
 - 4.2 Generator-based for loop
 - 4.3 Syntax sugar for sink calls (optional)
 - 4.4 `std::ranges` support (future paper)
- 5 Open questions
 - 5.1 Spelling of the generator function
 - 5.2 `auto` in for loop
 - 5.3 `std::stacktrace` and `__func__`
 - 5.4 Exceptions in for body and try-catch in operator `for`
 - 5.5 `co_await/co_yield/co_return` in for body
 - 5.6 Return type of sinks

§ 1 Abstract

We propose language support for internal iteration in a for loop by adding a generator-based for loop. Instead of using `begin()/end()`, it transforms the body of the for loop into a lambda and passes it to an overloaded `operator()`. The lambda will then be invoked with each element of the range.

```
struct generator123
{
    auto operator()(auto&& sink) const
    {
        std::control_flow flow;

        // With P2561: sink(1)??;
        flow = sink(1);
        if (!flow) return flow;

        // With P2561: sink(2)??;
        flow = sink(2);
        if (!flow) return flow;

        return sink(3);
    }
};

for (int x : generator123())
{
    std::print("{}\n", x);
    if (x == 2)
        break;
}
```

§ 2 Motivation

For complex ranges, iterators are a bit annoying to write: There is an awkward split between returning the value in `operator*` and computing the next one in `operator++`, and any state needs to be awkwardly stored in the iterator to turn a state machine.

C++20 coroutines and C++23 `std::generator` make it a lot more convenient. Compare e.g. the implementation of `std::views::filter`, `std::views::join` or the proposed `std::views::concat` with a generator-based implementation:

```
template <typename Rng, typename Predicate>
auto filter(Rng&& rng, Predicate predicate) -> std::generator<...>
{
    for (auto&& elem : rng)
        if (predicate(elem))
            co_yield std::forward<decltype(elem)>(elem);
}
```

```

template <typename Rng>
auto join(Rng&& rng_of_rng) -> std::generator<...>
{
    for (auto&& rng : rng_of_rng)
        co_yield std::ranges::elements_of(std::forward<decltype(rng)>
            (rng));
}

template <typename ... Rng>
auto concat(Rng&& ... rng ) -> std::generator<...>
{
    ((co_yield std::ranges::elements_of(std::forward<decltype(rng)>
        (rng))), ...);
}

```

However, ranges implemented using `co_yield` have three disadvantages.

§ 2.1 Performance of `std::generator`, coroutines, and iterators

The coroutine transformation requires heap allocation, splitting one function into multiple, and exception machinery. This adds overhead compared to the iterator version or a hand-written version.

In the simple case of benchmarking `std::views::iota` against a coroutine-based version, the latter is 3x slower on GCC. Note that the coroutine uses `SimpleGenerator` not `std::generator`. `SimpleGenerator` already optimizes heap allocations by using a pre-allocated buffer and terminates on an unhandled exception. The benchmark is available on [quick-bench](#). On clang, coroutines are better optimized, but they are still sometimes slower than a callback-based version, especially when the range is small and/or the computation per element cheap.

Just writing manual iterators instead isn't a solution either. Common state of the range (like `filter_view`'s predicate) is stored in the view, which the iterator accesses via pointer. As compilers can rarely proof non-aliasing with pointers, the state needs to be repeatedly reloaded for every access. The only way to get full control over performance is to use neither iterators nor coroutines, which is a shame.

Yes, in principle compilers could optimize coroutines and iterators better, but there is a difference between having something that is only fast because of optimizers and something that is fast by design. Only the latter is a true zero-overhead abstraction.

§ 2.2 Inability to `co_yield` from nested scope

Consider a simple tree type which is either a leaf storing an `int` or a `std::vector` of child nodes.

```

struct tree
{

```

```

using leaf = int;
std::variant<leaf, std::vector<tree>> impl;
};

```

Writing a coroutine-based generator that yields data cannot directly use `std::visit`, as `co_yield` needs to be in the top-level scope:

```

std::generator<int> tree_data(const tree& t)
{
    std::visit(overloaded([&](int data) {
        co_yield data; // error
    }, [&](const std::vector<tree>& children) {
        for (auto& child : children)
            co_yield std::ranges::elements_of(tree_data(child)); //
        error
    }), t.impl);
}

```

Every single lambda needs to be turned into a generator as well, so we can finally yield its elements:

```

std::generator<int> tree_data(const tree& t)
{
    auto sub =
        std::visit(overloaded([&](int data) -> std::generator<int> {
            co_yield data;
        }, [&](const std::vector<tree>& children) -> std::generator<int> {
            for (auto& child : children)
                co_yield std::ranges::elements_of(tree_data(child));
        }), t.impl);
    co_yield std::ranges::elements_of(sub);
}

```

This particular problem can also be solved using pattern matching, but the underlying limitation remains: implementing a coroutine generator cannot use “normal” helper functions as you cannot use `co_yield` in them; they need to be turned into generators as well. If you can’t control the helper functions, such as the case of standard library algorithms, you simply can’t use them.

§ 2.3 Limitation to a single type

As specified, `std::generator` only produces a single type, so writing a generator of `std::tuple` is not possible. Note that this is only a limitation of `std::generator` and the iterator interface it implements, not of coroutines itself.

§ 3 Generator Range Idiom

In the [think-cell-library](#), this problem is solved using a concept of *generator ranges*. It is in some aspects even weaker than an input range – it can only be iterated over, and once the iteration is stopped, it cannot resume later on. However, it is enough for algorithms

that require only a single loop, like `std::ranges::for_each`, `std::ranges::copy`, or `std::ranges::fold_left` (plus variants and fold-like algorithms like `std::ranges::any_of`).

A generator range is implemented as a type with an overloaded `operator()` that takes another callable, a sink. It will then use internal iteration invoking the sink with each argument. Early exit (i.e. `break`) is possible by returning `tc::break_` from the sink, which the `operator()` then detects, stops iteration, and forwards the result.

Range adapters like `filter()`, `join()` or `concat()` are still easy to write:

```
template <typename Rng, typename Predicate>
auto filter(Rng&& rng, Predicate predicate)
{
    // Returned lambda is a generator range.
    return [=](auto sink) {
        // rng here is an iterator range, not a generator range.
        // With this proposal, it can be either, as the range-based for
        loop will work with both.
        // Without it, generator ranges would need to be handled
        specially.
        for (auto&& elem : rng)
            if (predicate(elem))
            {
                auto result = sink(std::forward<decltype(elem)>(elem));
                if (result == tc::break_)
                    return result;
            }

        return tc::continue_;
    };
}
```

However, we can also yield values from nested functions:

```
auto tree_data(const tree& t)
{
    return [&](auto sink) {
        auto flow =
            std::visit(overloaded([&](int data) {
                // Forward break/exit.
                return sink(data);
            }, [&](const std::vector<tree>& children) {
                for (auto& child : children)
                {
                    auto flow = tree_data(child)(sink);
                    if (flow == tc::break_)
                        // Forward early break and do actually break.
                        return flow;
                }
                return tc::continue_;
            })), t.impl);
        return flow;
    };
}
```

```
}  
}
```

As shown in the benchmark, the performance is comparable to the range implementation and in think-cell's experience often even faster.

§ 4 Proposed Design

The proposed design consists of two mandatory parts and optional syntax sugar.

§ 4.1 `std::control_flow`

```
namespace std  
{  
    /// A control flow tag type and object that means `continue`.  
    struct continue_t  
    {  
        // Empty.  
  
        constexpr operator std::true_type() const noexcept  
        {  
            return {};  
        }  
        constexpr std::false_type operator!() const noexcept  
        {  
            return {};  
        }  
  
        friend std::strong_ordering operator<==(continue_t, continue_t)  
            noexcept = default;  
    };  
    inline constexpr continue_t continue_;  
  
    /// A control flow tag type and object that means `break`.  
    struct [[nodiscard("need to forward break")]] break_t  
    {  
        // Empty.  
  
        constexpr operator std::false_type() const noexcept  
        {  
            return {};  
        }  
        constexpr std::true_type operator!() const noexcept  
        {  
            return {};  
        }  
  
        friend std::strong_ordering operator<==(break_t, break_t) noexcept  
            = default;  
    };  
    inline constexpr break_t break_;
```

```

    /// A control flow object that can mean `continue`, `break` or some
    implementation-defined `break`-like state.
class [[nodiscard("need to forward control flow")]] control_flow
{
public:
    /// Create a control flow object that means `continue`.
    constexpr control_flow(continue_t) noexcept;
    constexpr control_flow() noexcept : control_flow(continue_) {}

    /// Create a control flow object that means `break`.
    constexpr control_flow(break_t) noexcept;

    /// Trivially copyable.

    /// Return `true` if the control flow means `continue`, `false`
    otherwise.
    constexpr explicit operator bool() const noexcept;

    constexpr friend bool operator==(control_flow, control_flow)
    noexcept;
    constexpr friend std::strong_ordering operator<=>(control_flow,
    control_flow) noexcept;
};
}

```

`std::control_flow` is a class that represents a break or continue result of a function. It is essentially a strongly-typed version of `bool`, where `true` means `continue_` and `false` means `break_`. However, an implementation may give it additional states such as `return` (which maps to `break`) or `goto label` (which also maps to `break`).

Note that `std::continue_` and `std::break_` are not just a named constant of type `std::control_flow`, but of a distinct tag type, which has an implicit conversion to `std::true/false_type` and a likewise constant operator!. This ensures that the common case of “always continue” or “always break” can be encoded in the type system and guarantee optimizations.

The standardization of `std::control_flow` is the bare minimum to enable writing generator ranges:

```

struct generator123
{
    // `sink` takes the type yielded by the generator range.
    // and returns `std::control_flow`, `std::continue_t`, or
    `std::break_t`.
    auto operator()(auto&& sink) const -> std::control_flow
    {
        std::control_flow flow;

        flow = sink(1);
        if (!flow) return flow;

        flow = sink(2);
        if (!flow) return flow;
    }
}

```

```

        return sink(3);
    }
};

template <typename Generator>
void use_generator_range(Generator&& generator)
{
    generator([](auto value) {
        std::print("{}\n", value);
        // Unconditionally continue generating.
        return std::continue_;
    });

    generator([](auto value) -> std::control_flow {
        // Exit early if we see the value 42.
        if (value == 42)
            return std::break_;

        std::print("{}\n", value);
        return std::continue_;
    });
}

```

The remaining features are “just” language support to make this idiom nicer.

§ 4.2 Generator-based for loop

Basic lowering of a generator-based for loop.

User code	Lowered code
<pre> for (T binding : object) { body } </pre>	<pre> { auto __body = [&](T&& __element) -> see- below { T binding = std::forward<T> (__element); body return std::continue_; }); auto __flow = object(__body); see-below } </pre>

Previously, the range-based for loop was always lowered to an *iterator-based for loop* if the type has `begin()` and `end()`. We propose that it can also be lowered to a *generator-based for loop* if the type has an `operator()` that accepts a callable and returns `std::control_flow`, `std::break_t` or `std::continue_t`. If the type has both `begin()` and `end()`, the generator-based version is preferred as it is expected to be faster.

The for loop is lowered to a call to `operator()` on the generator, passing it the body of the loop transformed into a lambda. That lambda takes the type specified in the binding of

the loop qualified with `&&`. Reference collapsing rules ensure that it is turned into an lvalue reference if necessary. The extra step is necessary as the binding of the `for`-loop might be a structured binding which is not allowed as a lambda argument. The lambda body then consists of the binding of the range-based `for` loop, the body of the loop, and a (potentially unreachable) `return std::continue_`.

The loop body is kept as-is, except for control flow statements which need to be translated:

- A top-level `continue`; is transformed to `return std::continue_`.
- A top-level `break`; is transformed to `return std::break_`.
- A `return`; is transformed to `return implementation-defined`, which has the same effect as `std::break_` but also causes the compiler to forward the return after the loop.
- A `return expr`; is transformed to something that evaluates `expr` and stores it somewhere, before a `return implementation-defined` which has the same effect as `std::break_` but also causes the compiler to forward the return after the loop.
- A `goto` that exits the range-based `for` loop is transformed to `return implementation-defined`, which has the same effect as `std::break_` but also causes the compiler to forward the `goto` after the loop.
- A `throw` is kept as-is.
- A `co_await`/`co_yield`/`co_return` is ill-formed (but [see below](#) for a discussion about that).

The return type of the lambda is `std::continue_t` or `std::break_t` if that is the only returned control-flow case, or `std::control_flow` otherwise. If the lambda exits with an exception or returns `std::break_` it must not be called again. The return value of the lambda must be forwarded through the `operator()`.

After the call to `operator()`, the compiler may insert additional code to handle a return or `goto` of the body, to ensure that control flow jumps to the desired location.

Example lowering.

User code	Lowered code
-----------	--------------

User code	Lowered code
<pre> for (int x : generator123()) { if (x == 0) continue; if (x == 2) break; std::printf("%d\n", x); } </pre>	<pre> { auto __body = [&](int&& __element) -> std::control_flow { int x = __element; if (x == 0) return std::continue_; if (x == 2) return std::break_; std::printf("%d\n", x); return std::continue_; }; auto __flow = generator123() (__body); (void)__flow; } </pre>

More example lowerings are given on [godbolt](#). Note that the compiler may do a more efficient job during codegen than what we can express in C++. In particular, `return expr` must directly construct the expression in the return slot to enable copy elision.

§ 4.3 Syntax sugar for sink calls (optional)

The implementation of a generator will necessarily have code that invoke the sink and does an early return:

```

auto flow = sink(value);
if (!flow) return flow;

```

This is annoying, but also precisely the pattern [P2561R1] sets out to solve with its error propagation operator. If adopted, we can instead write:

```

sink(value)??;

```

Alternatively, we could keep the association with coroutine generators and use a special `co_yield-into` statement:

```

co_yield(sink) value;
// or
co_yield(sink, value);
// or
co_yield[sink] value;
// or
co_yield<sink> value;

```

However, re-using `co_yield` in that way might result in parsing ambiguities with regular coroutine `co_yield`, so that particular syntax might be unfeasible.

§ 4.4 `std::ranges` support (future paper)

If there is interest in the feature, a separate paper will add support for generator ranges to the standard library. This includes concepts for generator ranges, turning views into generator ranges for better iteration performance, and adding support for generator ranges to single-loop range algorithms like `std::ranges::for_each`, `std::ranges::copy`, or `std::ranges::fold_left` (plus variants and fold-like algorithms like `std::ranges::any_of`).

§ 5 Open questions

§ 5.1 Spelling of the generator function

As proposed, the generator-based for loop is triggered when the range type has an `operator()` that takes a sink and returns `std::control_flow`, `std::break_t`, or `std::continue_t`. This spelling has the advantage that lambdas can directly be used as generators. For example, if the range adapters were updated to use generator ranges (like the ones in the `think-cell-library`), we could write code like this:

```
// Copy a C FILE to a buffer.
std::ranges::copy([&](auto&& sink) {
    std::control_flow flow;
    while (true)
    {
        auto c = std::fgetc(file);
        if (c == EOF)
            break;

        flow = sink(c);
        if (!flow)
            break;
    }
    return flow;
}, buffer);
```

However, we are open to a different spelling. An early draft of the paper used `operator` for instead of `operator()`, where `operator` for is a new overloadable operator. This can make it more obvious that a type has custom behavior when used with a range-based for loop.

Using `operator` for means that lambdas no longer work as-is and would need to be wrapped into a type that has an `operator` for which calls the lambda. That works, but is also a bit unnecessary. Alternatively, a lambda that takes a sink and returns a type like `std::control_flow` could automatically get an `operator` for overload. Either the language provides one as member, or the standard library provides a generic non-member overload for callables. At that point, we might as well use `operator()` again though.

§ 5.2 auto in for loop

What should happen if the for loop uses `auto` for the type of the loop variable?

```
for (auto x : generator123())
...
```

With the specification of `operator()` it is very difficult for the compiler to figure out what is the actual value type of the `operator()` – it has to look in the body to see what is passed to the callable. There is also no limitation to only one type; an implementation may invoke the sink with different types.

There are three approaches to avoid the deduction:

1. Make it ill-formed. When the user wants to use a generator-based for loop, they have to specify a type for the loop variable.
2. Infer the type from somewhere else, maybe some trait that has to specialized or a member typedef given. Note that we cannot use the return type of `operator()` as that is `std::control_flow` or related.
3. Turn the body of the for loop into a generic lambda with an `auto` parameter instead of a fixed type.
4. As above, but make the program ill-formed if the lambda would be instantiated with multiple different types. In a way, the `auto` parameter in that particular compiler-generated lambda then would be more like the `auto` in a return type or variable declaration: a way to name a specific, yet unknown type, and not a template.

Option 1 is annoying. Option 2 is not easy because the type of the range depends on the cv-ref qualifications of `*this`. `think-cell` uses `decltype()` of a member function call for that purpose:

```
struct container
{
    // Non-const containers yield `int&`.
    int& generator_output_type(); // no definition
    // const containers yield `const int&`.
    const int& generator_output_type() const; // no definition
};

template <typename T>
using generator_output_type = decltype(std::declval<T>().generator_output_type());
```

However, adding an undefined member function might be a bit too weird for the standard.

Option 3 has the side-effect of allowing iteration of types like `std::tuple`:

```
template <typename ... T>
struct tuple
{
    auto operator()(auto&& sink) const
    {
        return generator_impl(std::index_sequence_for<T...>{}, sink);
    }
    template <std::size_t ... Idx, typename Fn>
```

```

    auto generator_impl(std::index_sequence<Idx...>, Fn&& sink) const
    {
        std::common_type_t<decltype(sink(std::get<Idx>(*this)))...> flow;
        // && short-circuits once we have a break-like control flow.
        ((flow = sink(std::get<Idx>(*this))) && ...);
        return flow;
    }
};

...

for (auto elem : std::make_tuple(42, 3.14f, "hello"))
    std::print("{}\n", elem);

```

This is a different way of implementing expansion statements [P1306R1].

Even if EWG doesn't want to support multiple output types for `for`, the library idiom of using `operator()` naturally supports it. Limiting the idiom just for the sake of the language feature is wrong – we still might want to iterate tuples by calling `operator()` manually. It is better to just constrain the use in `for` but not the general spelling of the generator function, as done in option 4.

§ 5.3 `std::stacktrace` and `__func__`

What does the following code print?

```

void foo()
{
    for (int x : generator123())
    {
        std::cout << __func__ << ' ' << std::stacktrace::current() <<
            '\n';
    }
}

```

Does it print the name `foo` and the stack trace beginning at `foo`, as that is what it looks like syntactically, or does it report the true stack trace involving the compiler generated lambda and arbitrarily many calls caused by `operator()`?

The authors prefer the second choice where a stack trace reports the true stack trace and not the syntactic one.

§ 5.4 Exceptions in `for` body and `try-catch` in `operator for`

We expect that the following code will throw on the first iteration (assuming the range is `[1, 2, 3]`) and thus never call `g()` or `h()`.

```

for (int x : generator123())
{
    if (x == 1) throw 42;
}

```

```

    g();
}
h();

```

But what if we have a somewhat malicious `operator()` implementation that swallows all exceptions?

```

struct generator123
{
    auto operator()(auto&& sink) const
    {
        try
        {
            return sink(1);
        }
        catch (...) {}
    }
};

```

As specified, the exception propagates out of `sink()` and is then discarded by the `operator()`, so we will call `h()`.

There are three approaches here:

1. Do nothing. If someone writes a malicious `operator()`, that's on them.
2. Make exceptions thrown inside the generated lambda uncatchable until they've left `operator()`. Using some compiler magic, we set a flag or something on the exception which disables `try/catch`. Once the exception has left the `operator()`, the flag is reset. This makes it impossible to catch exceptions thrown by the sink until they're back in the syntactic scope.
3. Make exceptions thrown inside the generated lambda implicitly re-thrown when caught in `operator()`. Similar to 2, but `operator for` is allowed to see the exception, just not to swallow them. After a catch, exceptions are automatically re-thrown.

The problem with 2 is that a generator might change itself during iteration inside `operator()`. When an exception is thrown, it needs to detect that to restore its previous state. The problem with 3 is potential overhead caused by the extra exception machinery, which we don't want to pay only to guard against malicious actors. Keep in mind that the `operator()`, the call to the sink, and a `try/catch` surrounding the sink might be separated by an arbitrary amount of intermediate function calls, so it cannot be done statically. As such, the authors prefer option 1.

§ 5.5 `co_await/co_yield/co_return` in `for` body

When a user `co_awaits` or `co_yields` inside the body of a generator-based `for` loop, we have a problem since we're no longer inside a coroutine, but a compiler-generated lambda.

To handle that properly both the compiler generated lambda and the `operator()` needs to be turned into coroutines to properly propagate the `co_await`.

While that could work for the compiler-generated lambda, it would be problematic for the `operator()`, as we now sometimes need `co_await` on the sink. This problem relates to the general problem of coroutines with generic code, and is best solved by a general solution for conditional `co_await` in generic code.

We thus propose to make it ill-formed to use coroutine statements in the body of a generator-based `for` loop (at least for now). Alternatively, we could instead silently fallback to the traditional iterator-based `for` loop which does not suffer from this problem. This works nicely with generic code, but requires that the type also has `begin()` and `end()` (otherwise it is ill-formed again).

§ 5.6 Return type of sinks

The implementation in `think-cell` allows arbitrary return types for the sink, not just `std::control_flow` and related types. If a type is not a control flow type including `void`, it is treated as `std::continue_`. That way, if a sink is written by hand or wraps an existing function, if it doesn't want an early return, it doesn't need to do anything. Otherwise, it would need to add an unnecessary `return std::continue_`.

If a generator range is only used with the range-based `for` loop, this is not necessary as the compiler generated sink will always have the right type, but in `think-cell`'s experience where generators can be used with arbitrary algorithms and their hand-written lambdas, it is very convenient if the work of adjusting the return type is moved from the caller to the generator implementation.

Relaxing the return type complicates the implementation for sink calls, which now need to translate an arbitrary type (including `void`) to `std::continue_t` before branching. As such, it is useful to have a standardized helper function for that logic, e.g. a `std::control_flow::invoke(sink, value)`:

```
template <typename Sink, typename ... Args>
constexpr auto std::control_flow::invoke(Sink&& sink, Args&&... args)
{
    using result = std::invoke_result_t<Sink&&, Args&&...>;
    if constexpr (std::is_same_v<result, std::control_flow>
        || std::is_same_v<result, std::continue_t>
        || std::is_same_v<result, std::break_t>)
    {
        return std::invoke(std::forward<Sink>(sink), std::forward<Args>
            (args)...);
    }
    else
    {
        std::invoke(std::forward<Sink>(sink), std::forward<Args>
            (args)...);
        return std::continue_;
    }
}
```

```
}  
}
```

Alternatively, the selected syntax sugar could be specified to do the translation as well.

§ 6 References

[P1306R1] Andrew Sutton, Sam Goodrick, Daveed Vandevoorde. 2019-01-21. Expansion statements.

<https://wg21.link/p1306r1>

[P2561R1] Barry Revzin. 2022-10-11. An error propagation operator.

<https://wg21.link/p2561r1>